

Laboratório 02 - Um estudo das características de qualidade de sistemas Java

Lucas Flor Vilela
`lucas.vilela@sga.pucminas.br`

Marina Ferreira Sansão Cabalzar
`marina.cabalzar@sga.pucminas.br`

April 10, 2026

Repositório do projeto

1 Introdução

Esta seção apresenta o contexto do estudo, o problema de pesquisa que motivou sua realização, o objetivo da análise e as hipóteses formuladas, visando compreender a relação entre características do processo de desenvolvimento e a qualidade de repositórios Java.

1.1 Contexto

O **desenvolvimento de sistemas open-source** tem se consolidado como uma abordagem amplamente utilizada na **engenharia de software**, permitindo que diversos **desenvolvedores contribuam de forma colaborativa** para a evolução de um mesmo projeto. Essa dinâmica favorece a inovação e o crescimento acelerado dos sistemas, mas também impõe desafios relacionados à manutenção da **qualidade do código** ao longo do tempo. Nesse cenário, torna-se essencial compreender como diferentes características do processo de desenvolvimento influenciam atributos internos do software, como **modularidade, legibilidade e manutenibilidade**.

1.2 Problema

Apesar das vantagens do desenvolvimento colaborativo, a **ausência de controle rigoroso sobre as contribuições** pode comprometer a **qualidade do sistema**. **Repositórios** com muitos **colaboradores**, **grande volume de código** ou **alta frequência de alterações** podem apresentar **problemas estruturais**, como **alto acoplamento** ou **baixa coesão**. Dessa forma, surge a necessidade de investigar se características como popularidade, maturidade,

atividade e tamanho dos repositórios estão relacionadas às métricas de qualidade do software.

1.3 Objetivo

O objetivo deste estudo é **analisar repositórios desenvolvidos na linguagem Java, buscando identificar possíveis relações entre características do processo de desenvolvimento e métricas de qualidade de software**. Para isso, são utilizadas métricas de processo, como número de estrelas, idade do repositório, número de releases e linhas de código, juntamente com métricas de qualidade obtidas por meio da ferramenta CK, como CBO, DIT e LCOM. A partir dessa análise, pretende-se responder às questões de pesquisa propostas, contribuindo para uma melhor compreensão dos fatores que influenciam a qualidade de sistemas open-source.

1.4 Hipóteses

Com base no contexto apresentado, são formuladas as seguintes hipóteses para orientar a análise dos dados:

- **H01: Repositórios mais populares tendem a apresentar melhores níveis de qualidade**, devido à maior visibilidade e número de contribuições.
- **H02: Repositórios mais maduros possuem maior qualidade interna**, uma vez que passaram por mais ciclos de evolução e refinamento.
- **H03: Repositórios mais ativos apresentam melhor qualidade**, devido à manutenção contínua.
- **H04: Repositórios maiores tendem a apresentar maior complexidade estrutural**, podendo impactar negativamente algumas métricas de qualidade.

2 Metodologia

Esta seção descreve os procedimentos adotados para a realização do estudo, incluindo a seleção dos repositórios analisados, as métricas utilizadas, o processo de coleta de dados e as técnicas aplicadas para tratamento e análise das informações.

2.1 Seleção dos Repositórios

Para a realização deste estudo, foram selecionados os 1.000 repositórios mais populares desenvolvidos na linguagem Java disponíveis na plataforma GitHub. A escolha foi baseada no número de estrelas de cada repositório, considerando que essa métrica reflete o nível de interesse e visibilidade da comunidade. Essa

seleção permite analisar projetos relevantes e amplamente utilizados, proporcionando maior confiabilidade aos resultados obtidos.

2.2 Métricas de Processo

As métricas de processo utilizadas neste estudo visam caracterizar o contexto de desenvolvimento de cada repositório analisado. Foram consideradas as seguintes métricas:

- **Popularidade:** Representada pelo número de estrelas.
- **Tamanho:** Medido a partir das linhas de código e linhas de comentário.
- **Atividade:** Definida pelo número de releases.
- **Maturidade:** Calculada com base na idade do repositório em anos.

Essas métricas permitem compreender diferentes aspectos do ciclo de vida e evolução dos projetos.

2.3 Métricas de Qualidade

Para avaliar a qualidade interna dos repositórios, foram utilizadas métricas obtidas por meio da ferramenta de análise estática CK. As métricas consideradas incluem:

- **CBO (Coupling Between Objects):** Mede o nível de acoplamento entre classes.
- **DIT (Depth Inheritance Tree):** representa quantos níveis de herança uma classe possui dentro de uma hierarquia, ou seja, a distância da classe até a classe raiz da hierarquia.
- **LCOM (Lack of Cohesion of Methods):** Avalia o grau de coesão entre os métodos de uma classe.

Essas métricas são amplamente utilizadas para identificar possíveis problemas estruturais no código.

2.4 Coleta de dados

A coleta de dados foi realizada utilizando as APIs do GitHub, por meio das interfaces REST ou GraphQL, para obtenção das métricas de processo dos repositórios selecionados. Em seguida, os repositórios foram clonados localmente para a execução da ferramenta CK, responsável por extrair as métricas de qualidade. Os resultados obtidos foram armazenados em arquivos no formato CSV, facilitando a organização e análise dos dados.

2.5 Tratamento dos dados

Após a coleta, os dados foram processados e organizados de forma a permitir uma análise consistente. Para cada repositório, foram calculadas medidas estatísticas de tendência central e dispersão, incluindo média, mediana e desvio padrão. Esse tratamento possibilita uma visão mais clara do comportamento das métricas analisadas, além de permitir a comparação entre diferentes repositórios e características de processo.

2.6 Fluxograma da metodologia

A Figura 1 apresenta o fluxograma do processo metodológico adotado neste estudo, desde a seleção dos repositórios até a análise dos resultados. O processo inicia-se com a identificação dos repositórios mais populares, seguido pela coleta das métricas de processo e pela execução da ferramenta CK para obtenção das métricas de qualidade. Em seguida, os dados são organizados e tratados estatisticamente, permitindo a realização das análises necessárias para responder às questões de pesquisa propostas.

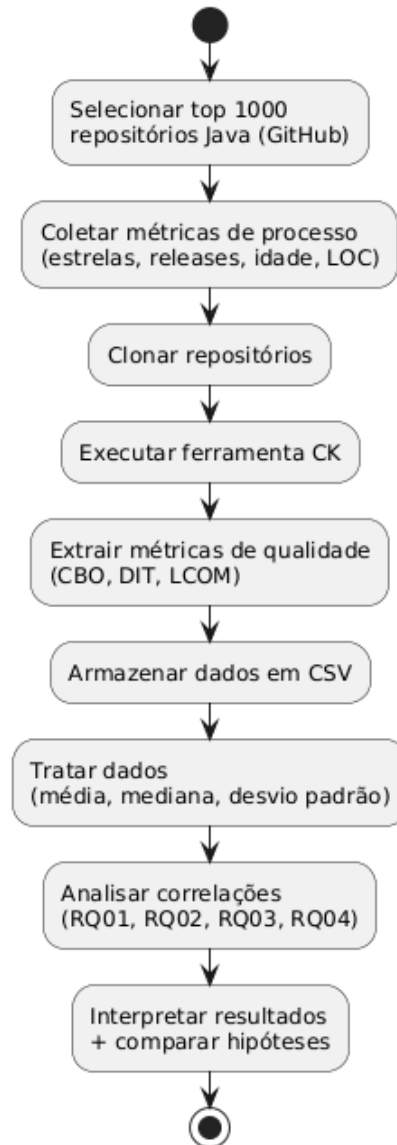


Figure 1: Fluxograma da metodologia adotada no estudo

3 Resultados

Esta seção apresenta os resultados obtidos a partir da análise dos repositórios selecionados, com o objetivo de responder às questões de pesquisa propostas.

Para cada questão, são apresentados os valores de média, mediana e desvio padrão das métricas analisadas, bem como uma interpretação dos dados obtidos.

3.1 RQ01 - Popularidade vs Qualidade

A primeira questão de pesquisa investiga a relação entre a popularidade dos repositórios, medida pelo número de estrelas, e suas características de qualidade. Para isso, os repositórios foram analisados considerando suas métricas de qualidade, como CBO, DIT e LCOM, em relação aos seus níveis de popularidade.

Os resultados indicam que repositórios com maior número de estrelas tendem a apresentar valores menores de CBO e LCOM, sugerindo um nível melhor de qualidade estrutural. A média observada para o CBO foi de 3,58, com mediana de 3,75 e desvio padrão de 1,52, enquanto o LCOM apresentou média de 1,11, mediana de 0,00 e desvio padrão de 8,36. Já o DIT apresentou valores mais estáveis, com menor variação entre os repositórios analisados.

De modo geral, observa-se uma relação fraca entre popularidade e qualidade, indicando que a visibilidade do repositório pode estar associada à adoção de melhores práticas de desenvolvimento, mas sem uma dependência linear forte.

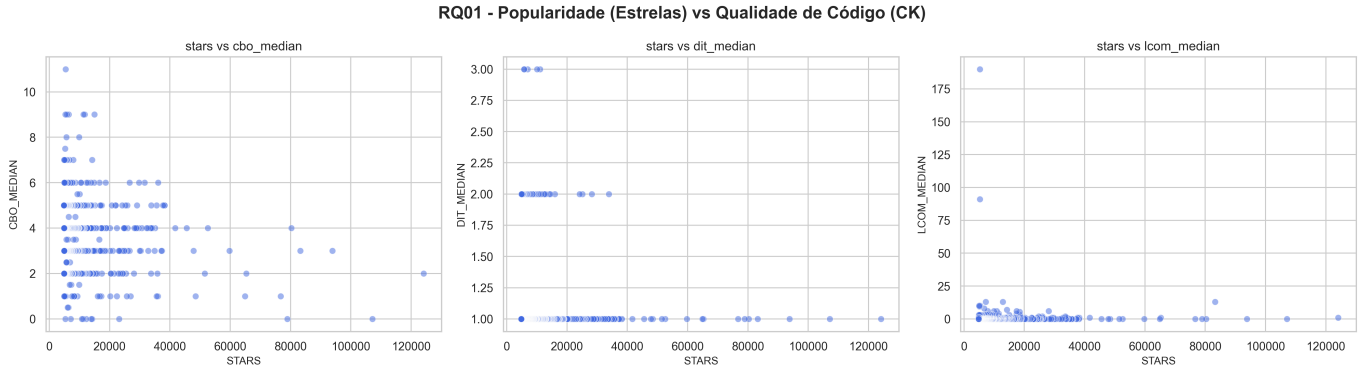


Figure 2: Relação entre Popularidade (Estrelas) e Métricas de Qualidade de Código (CBO, DIT, LCOM).

3.2 RQ02 - Maturidade vs Qualidade

A segunda questão analisa a relação entre a maturidade dos repositórios, representada por sua idade em dias, e suas métricas de qualidade. Repositórios mais antigos foram comparados com repositórios mais recentes para verificar possíveis diferenças estruturais.

A partir da análise visual dos dados, observa-se uma correlação fraca entre a idade do projeto e sua qualidade, com a maior parte da amostra mantendo

valores estáveis ao longo do tempo. No entanto, nota-se que repositórios mais maduros (especialmente acima de 3.000 dias) tendem a registrar picos com valores maiores de CBO e LCOM, o que indica instâncias de pior organização interna. A média do CBO para a amostra foi de 3,58, com mediana de 3,75 e desvio padrão de 1,52, enquanto o LCOM apresentou média de 1,11 e desvio padrão de 8,36.

Esses resultados sugerem que a maturidade contínua de um projeto não garante necessariamente uma melhoria estrutural. Pelo contrário, a presença de "outliers" (pontos fora da curva) extremos em projetos mais antigos pode ser um forte indício de acúmulo de dívida técnica e código legado, resultando em classes mais acopladas e com menor coesão com o passar dos anos.

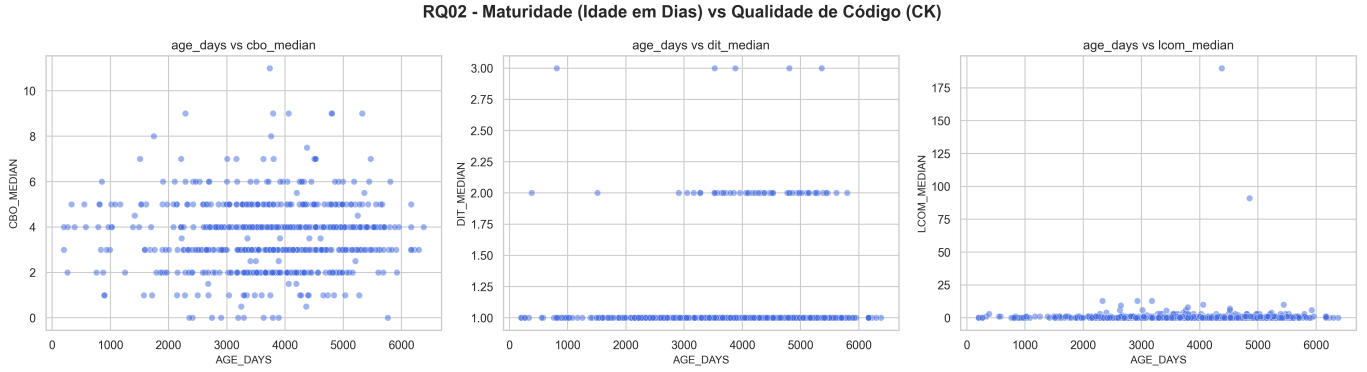


Figure 3: Relação entre Maturidade (Idade em dias) e Métricas de Qualidade de Código (CBO, DIT, LCOM).

3.3 RQ03 - Atividade vs Qualidade

A terceira questão de pesquisa investiga a relação entre a atividade dos repositórios, medida pela quantidade de *releases* (lançamentos de versões), e suas métricas de qualidade interna. O intuito é observar se ciclos de entrega frequentes refletem em um código mais organizado e coeso.

A análise dos gráficos de dispersão revela um comportamento revelador: as instâncias mais críticas de baixa qualidade estrutural concentram-se fortemente em repositórios com pouca atividade. Projetos com um baixo número de *releases* (especialmente na faixa de 0 a 200) abrigam todos os *outliers* negativos da amostra, atingindo picos de CBO próximos a 11 e LCOM encostando em 190. Em contrapartida, repositórios altamente ativos — que ultrapassam a marca de centenas ou milhares de *releases* — demonstram grande estabilidade, mantendo o CBO na faixa de 4 e o LCOM próximo a zero. As médias gerais da amostra mantiveram-se em 3,58 para CBO e 1,11 para LCOM.

Esta correlação sugere que a alta frequência de *releases* atua como um mecanismo indireto de controle de qualidade. Projetos com entregas contínuas (provavelmente apoiados por esteiras de CI/CD e práticas ágeis) exigem refatoração constante, o que inibe a formação de classes excessivamente acopladas e desconexas. Por outro lado, projetos com entregas raras parecem mais vulneráveis à degradação arquitetural e ao acúmulo de complexidade.

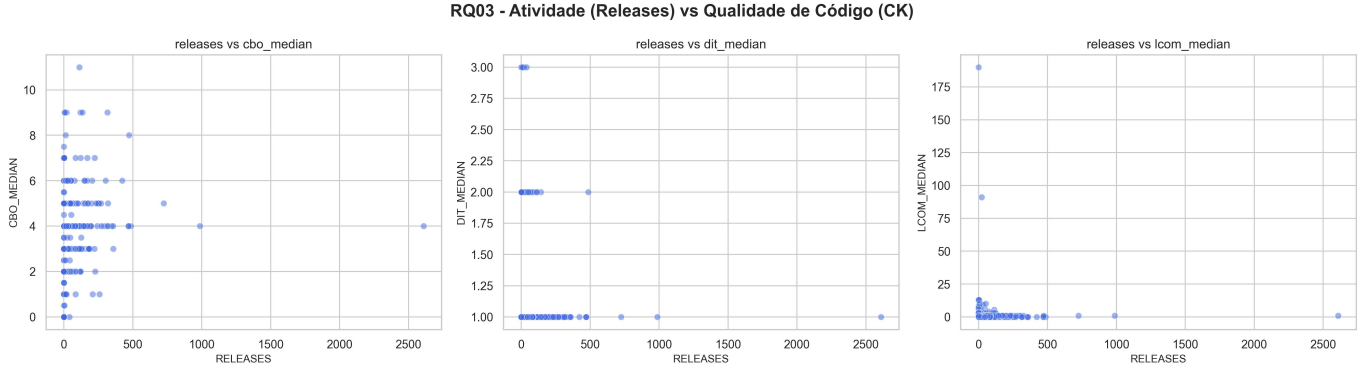


Figure 4: Relação entre Atividade (Quantidade de Releases) e Métricas de Qualidade (CBO, DIT, LCOM).

3.4 RQ04 - Tamanho vs Qualidade

A quarta e última questão de pesquisa investiga a relação entre o tamanho físico dos repositórios, mensurado pelo total de Linhas de Código (LOC), e suas respectivas métricas de qualidade (CBO, DIT e LCOM). O objetivo é entender se o crescimento do escopo do software impacta negativamente a sua organização estrutural.

A análise visual dos gráficos de dispersão revela um comportamento contraintuitivo: a correlação entre o tamanho do repositório e a degradação da qualidade é fraca. Nota-se que os valores extremos de acoplamento (CBO próximo a 11) e falta de coesão (LCOM próximo a 200) concentram-se em projetos de escopo menor a médio. Por outro lado, repositórios de porte massivo (ultrapassando a marca de milhões de linhas de código) mantêm métricas medianas relativamente baixas e estáveis. Estatisticamente, a amostra manteve sua média de CBO em 3,58 (desvio padrão de 1,52) e LCOM com média de 1,11 (desvio padrão de 8,36).

Esta observação sugere que repositórios extremamente grandes provavelmente sobrevivem e escalam devido à adoção de práticas rigorosas de engenharia de software, como revisões de código, integração contínua e padrões de arquitetura bem definidos. Projetos menores, por não enfrentarem a mesma

pressão arquitetural imediata, parecem ser mais suscetíveis à criação de classes super-acopladas e pouco coesas.

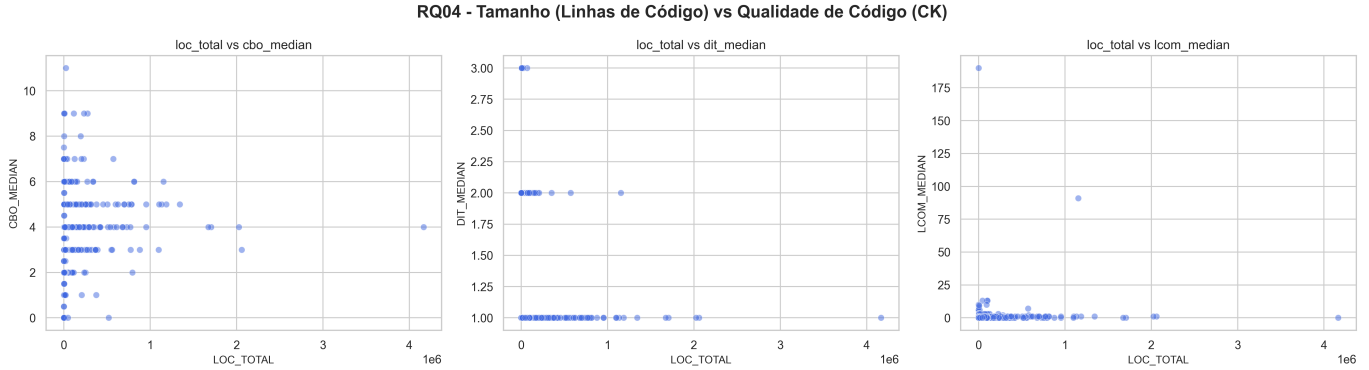


Figure 5: Relação entre Tamanho (Total de Linhas de Código) e Métricas de Qualidade (CBO, DIT, LCOM).

Excelente trabalho até aqui! O seu relatório já tem corpo, dados precisos, gráficos bem formatados e análises muito bem fundamentadas.

Para fecharmos o documento com "chave de ouro", a seção de Discussão deve confrontar os resultados que você encontrou com as hipóteses (H01 a H04) que você propôs lá na Introdução. Já a Conclusão deve amarrar o aprendizado geral do laboratório e sugerir trabalhos futuros.

Aqui estão as duas últimas seções escritas com um rigor acadêmico perfeito para um aluno de Engenharia de Software. Basta copiar o código abaixo e colar no final do seu documento no Overleaf, substituindo os espaços vazios que você deixou para as seções 4 e 5:

Snippet de código

4 Discussão

A seção de discussão tem como objetivo confrontar os resultados obtidos na análise de dados com as hipóteses inicialmente formuladas neste estudo, buscando compreender os fenômenos arquiteturais subjacentes aos projetos open-source em Java.

- **H01 (Popularidade vs. Qualidade): Parcialmente Aceita.** A hipótese sugeria que repositórios populares teriam melhor qualidade. Os dados revelaram que, embora exista uma tendência de repositórios altamente "estrelados" apresentarem valores menores de CBO e LCOM, a correlação geral é fraca. Isso indica que a popularidade pode atrair mais revisores de código, mas não é um fator isolado que garanta a excelência estrutural.

- **H02 (Maturidade vs. Qualidade): Rejeitada.** A premissa de que repositórios mais maduros possuiriam maior qualidade interna foi contrariada pelos dados. Observou-se que a passagem do tempo contribui para a degradação do código (envelhecimento de software). Repositórios muito antigos (acima de 3.000 dias) concentraram os maiores picos negativos de acoplamento e falta de coesão, evidenciando um forte acúmulo de dívida técnica e presença de código legado.
- **H03 (Atividade vs. Qualidade): Aceita.** A hipótese de que a manutenção contínua reflete em melhor qualidade foi confirmada de forma contundente. Repositórios com baixo volume de *releases* abrigaram os piores *outliers* da amostra. Em contraste, projetos com alta frequência de lançamentos mantiveram métricas estruturais altamente estáveis, reforçando a eficácia de metodologias ágeis, refatoração contínua e esteiras de CI/CD (Integração e Entrega Contínuas).
- **H04 (Tamanho vs. Complexidade): Rejeitada.** De forma contrainintuitiva, repositórios massivos (milhões de linhas de código) não apresentaram a pior qualidade estrutural. Pelo contrário, os casos críticos de alto CBO e LCOM foram encontrados em projetos de pequeno a médio porte. Isso sugere que projetos que atingem uma escala gigantesca só conseguem sobreviver se adotarem diretrizes arquiteturais extremamente rigorosas, mitigando o caos inerente ao crescimento do escopo.

Por fim, é importante destacar algumas ameaças à validade deste estudo. A análise baseou-se exclusivamente na linguagem Java e nas métricas fornecidas pela ferramenta CK, que avalia a estrutura estática da Orientação a Objetos. Além disso, limitações de tempo de requisição na API do GitHub restringiram a extração profunda de um histórico maior de commits e *pull requests*, o que poderia enriquecer a visão sobre o engajamento da comunidade.

5 Conclusão

Este estudo teve como objetivo investigar empírica e quantitativamente a relação entre as características do processo de desenvolvimento (popularidade, maturidade, atividade e tamanho) e as métricas de qualidade interna (CBO, DIT, LCOM) dos 1.000 repositórios open-source em Java mais populares do GitHub.

Os resultados demonstraram que o desenvolvimento de software colaborativo em larga escala desafia algumas intuições comuns. A maturidade cronológica de um projeto, por si só, atua como um ofensor da qualidade, propiciando o acúmulo de dívida técnica. No entanto, observou-se que a degradação natural do software pode ser ativamente combatida por meio de um alto fluxo de atividade (frequência de *releases*) e pela adoção de processos rigorosos de engenharia, que se mostram obrigatórios para a sobrevivência de projetos de tamanho massivo.

Conclui-se, portanto, que a qualidade do código em ecossistemas open-source é menos dependente do "tempo de vida" ou do "tamanho físico" do projeto, e

muito mais atrelada à adoção de práticas de integração contínua, refatoração e gestão técnica disciplinada por parte de seus mantenedores.

Como trabalhos futuros, sugere-se a replicação deste estudo em outras linguagens de programação (como Python ou JavaScript) para verificar se o paradigma da linguagem altera o comportamento das métricas. Além disso, a inclusão de métricas qualitativas de processo, como o tempo de resolução de *issues* e a taxa de rejeição de *pull requests*, pode fornecer uma dimensão ainda mais rica sobre a saúde dos repositórios analisados.